Project Brief: The "*Sugar Trap*" Market Gap Analysis

What I'm Building

I am working as a data analyst for Helix CPG Partners, a strategic food and beverage consultancy. The client is a global snack manufacturer preparing to launch a new healthy snacking product line.

The core business problem is that the client believes the snack market is oversaturated with high-sugar, high-fat products, but they need data to prove it, and more importantly, to identify exactly where the gaps are.

Story 1: Data Ingestion & "The Clean Up"

Cleaning the dataset that removes products with erroneous nutritional information, so that my analysis is not skewed by bad data entry.

In [120…

```
# libraries and modules needed for the project
import pandas as pd
import numpy as np
import os
import matplotlib.pyplot as plt
import seaborn as sns
import re
```

In [121…

```
#downloading the Open Food Facts dataset
!wget https://static.openfoodfacts.org/data/en.openfoodfacts.org.products.csv.gz
```

```
--2026-04-03 23:09:31--  https://static.openfoodfacts.org/data/en.openfoodfacts.org.products.c
sv.gz
Resolving static.openfoodfacts.org (static.openfoodfacts.org)... 213.36.253.214
Connecting to static.openfoodfacts.org (static.openfoodfacts.org)|213.36.253.214|:443... conne
cted.
HTTP request sent, awaiting response... 302 Moved Temporarily
Location: https://openfoodfacts-ds.s3.eu-west-3.amazonaws.com/en.openfoodfacts.org.products.cs
v.gz [following]
--2026-04-03 23:09:31--  https://openfoodfacts-ds.s3.eu-west-3.amazonaws.com/en.openfoodfacts.
org.products.csv.gz
Resolving openfoodfacts-ds.s3.eu-west-3.amazonaws.com (openfoodfacts-ds.s3.eu-west-3.amazonaw
s.com)... 3.5.204.182, 3.5.206.20
Connecting to openfoodfacts-ds.s3.eu-west-3.amazonaws.com (openfoodfacts-ds.s3.eu-west-3.amazo
naws.com)|3.5.204.182|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1246002034 (1.2G) [application/gzip]
Saving to: ‘en.openfoodfacts.org.products.csv.gz.3’

en.openfoodfacts.or 100%[===================>]   1.16G  25.5MB/s    in 49s

2026-04-03 23:10:21 (24.2 MB/s) - ‘en.openfoodfacts.org.products.csv.gz.3’ saved [1246002034/1
246002034]
```

In [122…

```
os.listdir()
```

Out[122…

```
['.config',
 'cleaned_food_data.csv',
 'en.openfoodfacts.org.products.csv.gz.1',
 'en.openfoodfacts.org.products.csv.gz.3',
 'AmaliTech_Capstone_Project.pdf',
 'AmaliTech_Capstone_Project.ipynb',
 'en.openfoodfacts.org.products.csv.gz',
 '.ipynb_checkpoints',
 'en.openfoodfacts.org.products.csv.gz.2',
 'sample_data']
```

In [123…

```
# reading the csv file and correctly resetting the index outside the function call
food_data = pd.read_csv("en.openfoodfacts.org.products.csv.gz", header=0, sep='\t', nrows=500
```

In [124…

```
food_data
```

Out[124…

	code	url	creator	created_t	created_datetime	last
0	54	http://world-en.openfoodfacts.org/product/0000...	kiliweb	1582569031	2020-02-24T18:30:31Z	
1	63	http://world-en.openfoodfacts.org/product/0000...	kiliweb	1673620307	2023-01-13T14:31:47Z	
2	114	http://world-en.openfoodfacts.org/product/0000...	kiliweb	1580066482	2020-01-26T19:21:22Z	
3	431	http://world-en.openfoodfacts.org/product/0000...	kiliweb	1714301712	2024-04-28T10:55:12Z	
4	105	http://world-en.openfoodfacts.org/product/0000...	kiliweb	1572117743	2019-10-26T19:22:23Z	
...	...	...	...	...	...	
499995	88573220018	http://world-en.openfoodfacts.org/product/0088...	kiliweb	1658369691	2022-07-21T02:14:51Z	
499996	88573331004	http://world-en.openfoodfacts.org/product/0088...	swipe-studio	1608767681	2020-12-23T23:54:41Z	
499997	88573402421	http://world-en.openfoodfacts.org/product/0088...	waistline-app	1750609917	2025-06-22T16:31:57Z	
499998	88573444445	http://world-en.openfoodfacts.org/product/0088...	foodless	1643224572	2022-01-26T19:16:12Z	
499999	88573800012	http://world-en.openfoodfacts.org/product/0088...	kiliweb	1626831085	2021-07-21T01:31:25Z	

500000 rows × 210 columns

In [125…

```
# displaying all the columns in the dataset to be able to select the needed categories
food_data.columns
```

Out[125…

```
Index(['code', 'url', 'creator', 'created_t', 'created_datetime',
       'last_modified_t', 'last_modified_datetime', 'last_modified_by',
       'last_updated_t', 'last_updated_datetime',
       ...
       'choline_100g', 'phylloquinone_100g', 'beta-glucan_100g',
       'inositol_100g', 'carnitine_100g', 'sulphate_100g', 'nitrate_100g',
       'acidity_100g', 'carbohydrates-total_100g', 'water_100g'],
      dtype='object', length=210)
```

In [126…

```
for column in food_data.columns:
  print(column)
```

```
code
url
creator
created_t
created_datetime
last_modified_t
last_modified_datetime
last_modified_by
last_updated_t
last_updated_datetime
product_name
abbreviated_product_name
generic_name
quantity
packaging
packaging_tags
packaging_en
packaging_text
brands
brands_tags
brands_en
categories
categories_tags
categories_en
origins
origins_tags
origins_en
manufacturing_places
manufacturing_places_tags
labels
labels_tags
labels_en
emb_codes
emb_codes_tags
first_packaging_code_geo
cities
cities_tags
purchase_places
stores
countries
countries_tags
countries_en
ingredients_text
ingredients_tags
ingredients_analysis_tags
allergens
allergens_en
traces
traces_tags
traces_en
serving_size
serving_quantity
no_nutrition_data
additives_n
additives
additives_tags
additives_en
nutriscore_score
nutriscore_grade
nova_group
pnns_groups_1
pnns_groups_2
food_groups
food_groups_tags
food_groups_en
states
```

```
states_tags
states_en
brand_owner
environmental_score_score
environmental_score_grade
nutrient_levels_tags
product_quantity
owner
data_quality_errors_tags
unique_scans_n
popularity_tags
completeness
last_image_t
last_image_datetime
main_category
main_category_en
image_url
image_small_url
image_ingredients_url
image_ingredients_small_url
image_nutrition_url
image_nutrition_small_url
energy-kj_100g
energy-kcal_100g
energy_100g
energy-from-fat_100g
fat_100g
saturated-fat_100g
butyric-acid_100g
caproic-acid_100g
caprylic-acid_100g
capric-acid_100g
lauric-acid_100g
myristic-acid_100g
palmitic-acid_100g
stearic-acid_100g
arachidic-acid_100g
behenic-acid_100g
lignoceric-acid_100g
cerotic-acid_100g
montanic-acid_100g
melissic-acid_100g
unsaturated-fat_100g
monounsaturated-fat_100g
omega-9-fat_100g
polyunsaturated-fat_100g
omega-3-fat_100g
omega-6-fat_100g
alpha-linolenic-acid_100g
eicosapentaenoic-acid_100g
docosahexaenoic-acid_100g
linoleic-acid_100g
arachidonic-acid_100g
gamma-linolenic-acid_100g
dihomo-gamma-linolenic-acid_100g
oleic-acid_100g
elaidic-acid_100g
gondoic-acid_100g
mead-acid_100g
erucic-acid_100g
nervonic-acid_100g
trans-fat_100g
cholesterol_100g
carbohydrates_100g
sugars_100g
added-sugars_100g
```

sucrose_100g
glucose_100g
fructose_100g
galactose_100g
lactose_100g
maltose_100g
maltodextrins_100g
psicose_100g
starch_100g
polyols_100g
erythritol_100g
isomalt_100g
maltitol_100g
sorbitol_100g
fiber_100g
soluble-fiber_100g
polydextrose_100g
insoluble-fiber_100g
proteins_100g
casein_100g
serum-proteins_100g
nucleotides_100g
salt_100g
added-salt_100g
sodium_100g
alcohol_100g
vitamin-a_100g
beta-carotene_100g
vitamin-d_100g
vitamin-e_100g
vitamin-k_100g
vitamin-c_100g
vitamin-b1_100g
vitamin-b2_100g
vitamin-pp_100g
vitamin-b6_100g
vitamin-b9_100g
folates_100g
vitamin-b12_100g
biotin_100g
pantothenic-acid_100g
silica_100g
bicarbonate_100g
potassium_100g
chloride_100g
calcium_100g
phosphorus_100g
iron_100g
magnesium_100g
zinc_100g
copper_100g
manganese_100g
fluoride_100g
selenium_100g
chromium_100g
molybdenum_100g
iodine_100g
caffeine_100g
taurine_100g
methylsulfonylmethane_100g
ph_100g
fruits-vegetables-legumes_100g
collagen-meat-protein-ratio_100g
cocoa_100g
chlorophyl_100g
carbon-footprint_100g

```
glycemic-index_100g
water-hardness_100g
choline_100g
phylloquinone_100g
beta-glucan_100g
inositol_100g
carnitine_100g
sulphate_100g
nitrate_100g
acidity_100g
carbohydrates-total_100g
water_100g
```

In [127…

```python
columns = [
    'product_name',
    'brands',
    'categories_en',
    'categories_tags',
    'main_category_en',
    'pnns_groups_1',
    'pnns_groups_2',
    'countries_en',
    'sugars_100g',
    'fat_100g',
    'saturated-fat_100g',
    'proteins_100g',
    'fiber_100g',
    'carbohydrates_100g',
    'salt_100g',
    'energy-kcal_100g',
    'nutriscore_score',
    'nutriscore_grade',
    'nova_group',
    'unique_scans_n',
    'ingredients_text',
]
```

In [128…

```python
food_data = food_data[columns]
food_data
```

Out[128…

	product_name	brands	categories_en	categories_tags	main_category_en	pnns_
0	Limonade artisanale a la rose	NaN	NaN	NaN	NaN	
1	M&M white	Fitpiggy	NaN	NaN	NaN	
2	Chocolate n3	Jeff de Bruges	NaN	NaN	NaN	
3	Pâte de fruits	NaN	NaN	NaN	NaN	
4	Paleta gran reserva - Sierra nevada-	AdvoCare	Beverages and beverages preparations,Beverages	en:beverages-and-beverages-preparations,en:bev...	Beverages	I
...	...	...	...	...	...	
499995	Seltzer	NaN	NaN	NaN	NaN	
499996	Beer	Shiner	NaN	NaN	NaN	
499997	Light Blonde	Shiner	NaN	NaN	NaN	
499998	Shiner Bock	NaN	NaN	NaN	NaN	
499999	Holiday Cheer	Shiner	NaN	NaN	NaN	

500000 rows × 21 columns

In [129…

```
# removing rows with missing values
food_data = food_data.dropna(subset=[
    'product_name',
    'sugars_100g',
    'fat_100g',
    'saturated-fat_100g',
    'proteins_100g',
    'fiber_100g',
    'carbohydrates_100g',
    'salt_100g',
    'energy-kcal_100g',
])

#removing rows where product_name is empty
food_data = food_data[food_data['product_name'].str.strip() != ""]
```

In [130…

```
#keeping only nutritional values (0-100g per 100g)
food_data = food_data[
    (food_data['sugars_100g'].between(0, 100)) &
    (food_data['proteins_100g'].between(0, 100)) &
    (food_data['fat_100g'].between(0, 100)) &
    (food_data['fiber_100g'].between(0, 100)) &
    (food_data['carbohydrates_100g'].between(0, 100)) &
    (food_data['salt_100g'].between(0, 100)) &
    (food_data['saturated-fat_100g'].between(0, 100))
]
```

In [131…

```
food_data
```

Out[131…

	product_name	brands	categories_en	categories_tags	main_category_en	pnns_
695	Keto & GF Granola	Cook & Baker Bali	NaN	NaN	NaN	
843	pasta	Grappa	Beverages and beverages preparations,Beverages	en:beverages-and-beverages-preparations,en:bev...	Beverages	
851	Dunnes Stores Irish Beef & Pork Quarter Pounders	Dunnes Stores	NaN	NaN	NaN	
862	Protein Plenty Tamarind Almonds	Itsu	NaN	NaN	NaN	
868	Eirn original curry Sauce	Eirn	Plant-based foods and beverages,Plant-based fo...	en:plant-based-foods-and-beverages,en:plant-ba...	Sauce-powder	
...	...	...	...	...	...	
499955	Garlic Original	Foster's Pickled Products	NaN	NaN	NaN	
499965	Asparagus Original	Foster's Pickled Products	NaN	NaN	NaN	
499982	Blackened Salmon With Couscous, Quinoa, & Veggies	Fusha	NaN	NaN	NaN	
499986	Blackened Chicken With Mac & Cheese	Marwan’s Street Grub	NaN	NaN	NaN	
499989	Swedish Meatballs	Marwan’s Kitchen - Street Grub	NaN	NaN	NaN	

63423 rows × 21 columns

In [132…

```
food_data.describe()
```

Out[132…

	sugars_100g	fat_100g	saturated-fat_100g	proteins_100g	fiber_100g	carbohydrates_100g	63
count	63423.000000	63423.000000	63423.000000	63423.000000	63423.000000	63423.000000	
mean	12.038820	11.635392	3.874285	8.678158	2.907247	33.915101	
std	17.341024	14.970220	6.351785	9.516378	4.899798	27.300066	
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	
25%	1.176471	1.180000	0.000000	2.941178	0.000000	8.666667	
50%	4.500000	5.300000	1.250000	6.670000	1.662500	27.000000	
75%	14.290000	17.648529	5.000000	10.800000	3.571429	57.140000	
max	100.000000	100.000000	100.000000	100.000000	100.000000	100.000000	

In [133…

```
food_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 63423 entries, 695 to 499989
Data columns (total 21 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   product_name        63423 non-null  object
 1   brands              49535 non-null  object
 2   categories_en       35977 non-null  object
 3   categories_tags     35977 non-null  object
 4   main_category_en    35977 non-null  object
 5   pnns_groups_1       63423 non-null  object
 6   pnns_groups_2       63423 non-null  object
 7   countries_en        63380 non-null  object
 8   sugars_100g         63423 non-null  float64
 9   fat_100g            63423 non-null  float64
 10  saturated-fat_100g  63423 non-null  float64
 11  proteins_100g       63423 non-null  float64
 12  fiber_100g          63423 non-null  float64
 13  carbohydrates_100g  63423 non-null  float64
 14  salt_100g           63423 non-null  float64
 15  energy-kcal_100g    63423 non-null  float64
 16  nutriscore_score    34100 non-null  float64
 17  nutriscore_grade    63423 non-null  object
 18  nova_group          32202 non-null  float64
 19  unique_scans_n      28094 non-null  float64
 20  ingredients_text    34471 non-null  object
dtypes: float64(11), object(10)
memory usage: 10.6+ MB
```

Story 2: The Category Wrangler

Grouping products into readable high-level categories, so that I don't have to look at 10,000 unique, messy tags.

In [134…

```
#filliing the NaN values in the categories_en  with an empty string to avoid errors during str
food_data['categories_en'] = food_data['categories_en'].fillna('')
```

In [135…

```
#showing the different category tags
food_data['categories_tags'].unique()
```

Out[135…

```
array([nan, 'en:beverages-and-beverages-preparations,en:beverages',
       'en:plant-based-foods-and-beverages,en:plant-based-foods,en:condiments,en:spices,en:cu
rry-powder,en:powder,en:sauce-powder',
       ..., 'es:salsa-de-tomate-enlatado',
       'en:plant-based-foods-and-beverages-plant-based-foods-condiments-spices-curry-curry-po
wder',
       'en:plant-based-foods-and-beverages,en:plant-based-foods,en:cereals-and-potatoes,en:ce
reals-and-their-products,en:meals,en:pasta-dishes,en:pastas,en:stuffed-pastas,en:ravioli,en:r
efrigerated-foods,en:meals-with-shellfish,en:refrigerated-meals,en:chinese-raviolis,en:japane
se-ravioli,en:meals-with-shrimps,fr:gyosas-aux-crevettes,fr:ravioli-a-la-crevette,en:raviolis
-aux-crevettes'],
      dtype=object)
```

In [136…

```
# the categories function takes the category tags of a product and maps them to a simplified c
def categories(category_tags):
    # converts the input to a lowercase string to ensure consistent keyword matching across th
    tags = str(category_tags).lower()

    # snack sub categories
    if 'chocolate' in tags or 'chocolat' in tags:
        return 'Chocolate'

    elif any(tag in tags for tag in ['chips', 'crisps', 'popcorn', 'pretzel', 'crackers', 'to
        return 'Chips & Crisps'

    elif any(tag in tags for tag in ['salty', 'salted', 'craquelins', 'appetizers']):
        return 'Salty Snacks'

    elif any(tag in tags for tag in ['cookie', 'biscuit', 'wafer', 'cake', 'muffin', 'pastry'
        return 'Cookies & Cakes'

    elif any(tag in tags for tag in ['granola', 'protein-bar', 'energy-bar', 'cereal-bar', 'p
        return 'Bars & Granola'

    elif any(tag in tags for tag in ['nut', 'seed', 'peanut', 'almond', 'cashew', 'walnut', '
        return 'Nuts & Seeds'

    elif any(tag in tags for tag in ['confectionery', 'bonbon', 'candy', 'caramel', 'fudge',
        return 'Candies & Sweets'

    elif any(tag in tags for tag in ['beverage', 'drink', 'juice', 'water', 'coffee', 'tea',
        return 'Beverages'

    elif any(tag in tags for tag in ['dairy', 'milk', 'cheese', 'yogurt', 'yaourt', 'pudding'
        return 'Dairy'

    elif any(tag in tags for tag in ['fruit', 'vegetable', 'plant-based', 'legume', 'compote'
        return 'Fruits, Vegetables & Legumes'

    elif any(tag in tags for tag in ['grain', 'bread', 'pasta', 'rice', 'cereal', 'flour']):
        return 'Grains & Baked Goods'

    elif any(tag in tags for tag in ['meat', 'poultry', 'fish', 'seafood', 'beef', 'chicken',
        return 'Meat & Seafood'

    elif any(tag in tags for tag in ['meal', 'prepared', 'pizza', 'soup', 'stew', 'ravioli',
        return 'Prepared Meals'

    elif any(tag in tags for tag in ['oil', 'fat', 'butter', 'margarine']):
        return 'Fats & Oils'

    elif any(tag in tags for tag in ['sauce', 'spice', 'condiment', 'ketchup', 'mustard', 'pe
        return 'Sauces & Spices'
```

```
    else:
        return 'Other Snacks'
```

In [137…

```
#applying the cleaned categorization logic using the renamed function
food_data['primary_category'] = food_data['categories_tags'].apply(categories)
```

In [138…

```
food_data['primary_category'].value_counts().sort_index()
```

Out[138…

primary_category	count
Bars & Granola	272
Beverages	13858
Candies & Sweets	11
Chips & Crisps	1789
Chocolate	1505
Cookies & Cakes	7133
Dairy	1468
Fats & Oils	53
Fruits, Vegetables & Legumes	95
Grains & Baked Goods	590
Meat & Seafood	2079
Nuts & Seeds	1770
Other Snacks	29482
Prepared Meals	878
Salty Snacks	662
Sauces & Spices	1778

dtype: int64

In [139…

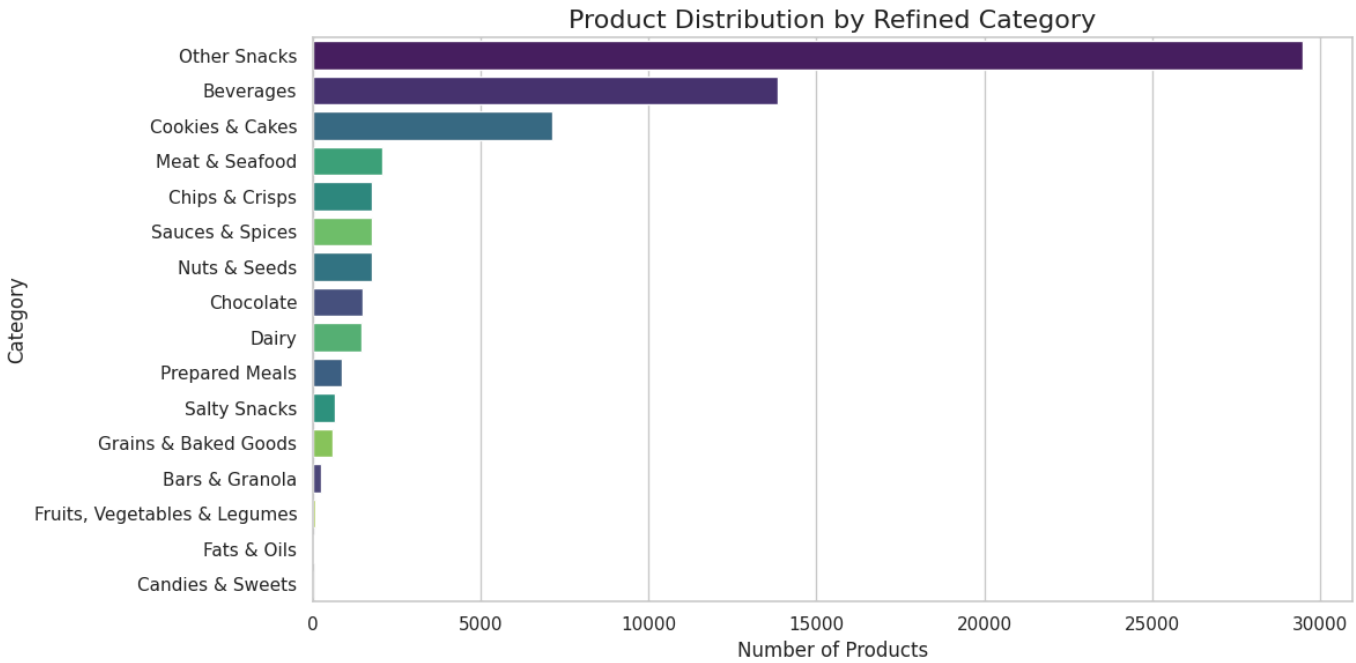
```
food_data[['product_name', 'primary_category']].head(10)
```

Out[139…

	product_name	primary_category
695	Keto & GF Granola	Other Snacks
843	pasta	Beverages
851	Dunnes Stores Irish Beef & Pork Quarter Pounders	Other Snacks
862	Protein Plenty Tamarind Almonds	Other Snacks
868	Eirn original curry Sauce	Beverages
895	Familjepack Schnitzel Frysta	Beverages
932	Crème oignon	Other Snacks
958	Coca cola	Beverages
1004	Tostadas	Beverages
1019	Omelette al gusto bacon	Other Snacks

In [140…

```
#visualizing the distribution of the categories
plt.figure(figsize=(12, 6))
sns.countplot(
    data=food_data,
    y='primary_category',
    order=food_data['primary_category'].value_counts().index,
    hue='primary_category',
    palette='viridis',
    legend=False
)

plt.title('Product Distribution by Refined Category', fontsize=16)
plt.xlabel('Number of Products', fontsize=12)
plt.ylabel('Category', fontsize=12)
plt.tight_layout()
plt.show()
```

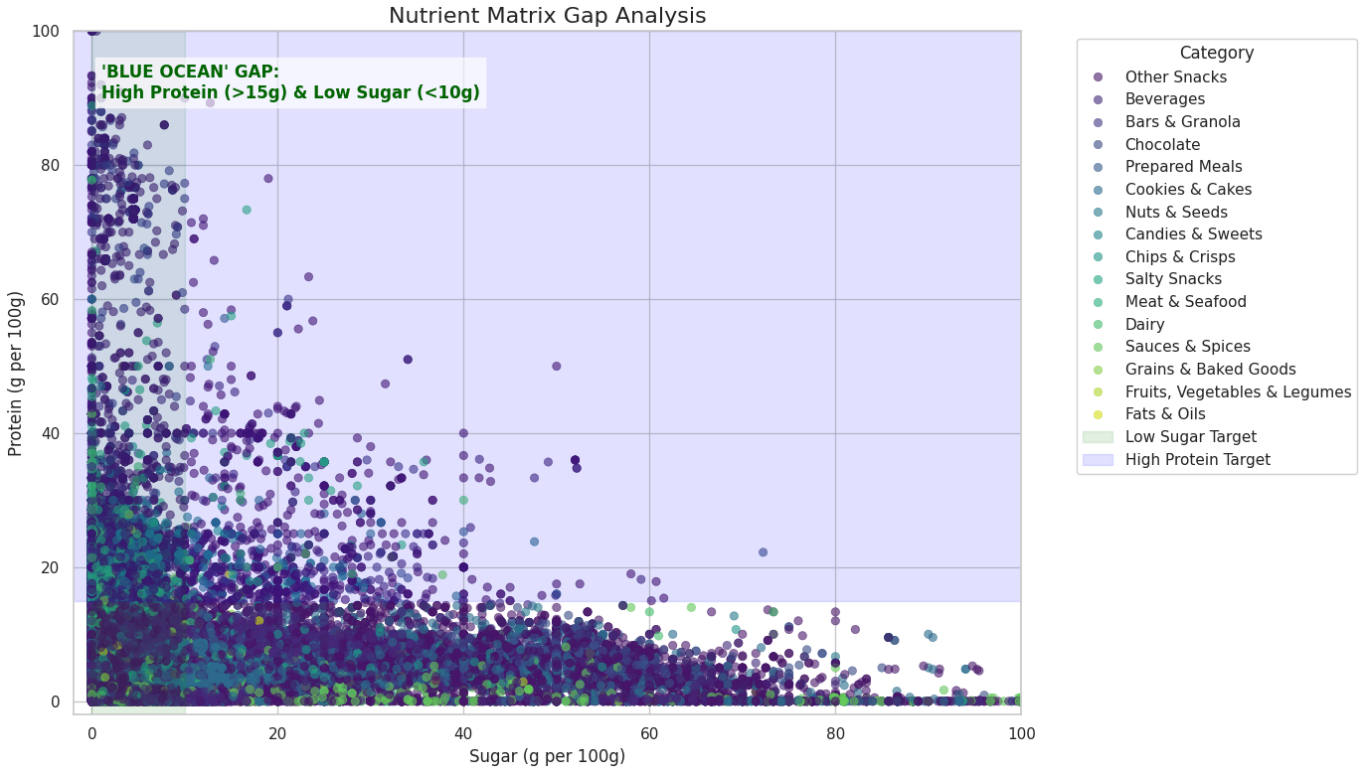


Story 3: The "Nutrient Matrix" Visualization

Creating a scatter Plot comparing Sugar (X-axis) vs. Protein (Y-axis) for different categories, so that I can visually spot where the products are clustered.

In [141…

```
# defining the target segments with lower-case variable names
low_sugar_threshold = 10
high_protein_threshold = 15

plt.figure(figsize=(14, 8))
sns.set_theme(style='whitegrid')

#creaating scatter plot
scatter = sns.scatterplot(
    data=food_data,
    x='sugars_100g',
    y='proteins_100g',
    hue='primary_category',
    alpha=0.6,
    edgecolor=None,
    palette='viridis'
)

#highlighting the empty quadrant (High Protein + Low Sugar)
plt.axvspan(0, low_sugar_threshold, color='green', alpha=0.1, label='Low Sugar Target')
plt.axhspan(high_protein_threshold, 100, color='blue', alpha=0.1, label='High Protein Target'

plt.text(1, 90, "'BLUE OCEAN' GAP:\nHigh Protein (>15g) & Low Sugar (<10g)",
         fontsize=12, color='darkgreen', fontweight='bold',
         bbox=dict(facecolor='white', alpha=0.7))

plt.title('Nutrient Matrix Gap Analysis', fontsize=16)
plt.xlabel('Sugar (g per 100g)', fontsize=12)
plt.ylabel('Protein (g per 100g)', fontsize=12)
plt.xlim(-2, 100)
plt.ylim(-2, 100)
plt.legend(title='Category', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```



In [142…

```
#calculating the stats
total_products = food_data[(food_data['sugars_100g'] < 10) & (food_data['proteins_100g'] > 15

print(f"Total products found in the 'Blue Ocean' quadrant: {len(total_products)}")

print("\nTop categories currently occupying this space:")
```

```
display(total_products['primary_category'].value_counts())

print("\nSample products in the target gap:")
display(total_products[['product_name', 'brands', 'sugars_100g', 'proteins_100g']].head(10))
```

```
Total products found in the 'Blue Ocean' quadrant: 8190

Top categories currently occupying this space:
```

primary_category	count
Other Snacks	4751
Meat & Seafood	1153
Dairy	709
Nuts & Seeds	665
Beverages	399
Bars & Granola	144
Salty Snacks	117
Cookies & Cakes	65
Grains & Baked Goods	60
Prepared Meals	41
Chips & Crisps	32
Chocolate	32
Sauces & Spices	20
Fats & Oils	2

dtype: int64

```
Sample products in the target gap:
```

	product_name	brands	sugars_100g	proteins_100g
695	Keto & GF Granola	Cook & Baker Bali	3.225806	19.677419
851	Dunnes Stores Irish Beef & Pork Quarter Pounders	Dunnes Stores	0.521739	15.652174
862	Protein Plenty Tamarind Almonds	Itsu	4.750000	27.500000
1019	Omelette al gusto bacon	Iberia	1.600000	71.000000
1157	Uncured Beef Hot Dogs	Good Chop	0.000000	17.543860
1162	Ribeye	Good Chop	0.000000	18.518845
1201	carte macdo	NaN	9.000000	17.000000
1239	Everything Nut Blend	nutri	3.571429	17.857143
1550	Lpm	NaN	0.000000	25.000000
1571	Tenders kfc	NaN	0.500000	21.000000

In [143…

```
# creating a market summary table with average sugar and protein metrics
category_nutrient_summary = food_data.groupby('primary_category').agg({'sugars_100g': 'mean',

# calculating Saturation_Score and add it to the summary table
```

```
category_nutrient_summary['Saturation_Score'] = (
    category_nutrient_summary['Total Count'] / category_nutrient_summary['Total Count'].max()
    category_nutrient_summary['Avg Sugar (g)'] / 100 * 0.3
)

# displaying the styled benchmark table with the new saturation score
category_nutrient_summary.style.background_gradient(cmap='RdYlGn_r', subset=['Avg Sugar (g)']
```

Out[143…

primary_category	Avg Sugar (g)	Avg Protein (g)	Total Count	Saturation_Score
Bars & Granola	7.005308	41.598873	272	0.027474
Meat & Seafood	1.876502	17.266874	2079	0.054992
Dairy	3.226358	15.352797	1468	0.044534
Nuts & Seeds	6.036145	13.072552	1770	0.060134
Other Snacks	12.414710	9.385985	29482	0.737244
Salty Snacks	3.687060	8.516106	662	0.026779
Grains & Baked Goods	3.482467	8.054733	590	0.024456
Beverages	8.924167	7.113822	13858	0.355807
Chips & Crisps	7.372209	6.891125	1789	0.064593
Chocolate	39.110490	6.688538	1505	0.153065
Prepared Meals	3.748853	6.308909	878	0.032093
Cookies & Cakes	18.950485	5.520013	7133	0.226212
Fruits, Vegetables & Legumes	12.693547	4.404538	95	0.040336
Fats & Oils	3.837149	3.771124	53	0.012770
Sauces & Spices	20.125687	2.175027	1778	0.102593
Candies & Sweets	38.714407	1.948207	11	0.116404

The saturation score was introduced to quantify market opportunity by combining product count and average sugar content into a single metric. This helps in identifying categories that are less saturated and potentially have higher growth potential for new, healthier products. It gives a more holistic view beyond just individual nutrient levels, providing strategic insight into market gaps.

Story 4: The Recommendation

My recommendation on what product the client should build.

Strategic Recommendation for Helix CPG Partners:

Based on the data derived from the Nutrient Matrix Gap Analysis in Story 3 and the Category Nutrient Summary Table, the biggest market opportunity is in Salty Snacks (specifically savory protein bars or high-protein crisps), specifically targeting products with more than 15g of protein and less than 10g of sugar.

5. Bonus User Story: The "Hidden Gem"

In [144…

```
# focusing on the 'High Protein' data defined earlier
high_protein = food_data[(food_data['proteins_100g'] > 15) & (food_data['sugars_100g'] < 10)]

# cleaning and flattening the ingredients text
ingredients_list = high_protein['ingredients_text'].dropna().str.lower()

# defining common protein sources to search for
protein_keywords = ['soy', 'soja', 'whey', 'lactoserum', 'milk', 'lait', 'egg', 'oeuf', 'pean
```

In [145…

```
found_sources = []
for text in ingredients_list:
    for source in protein_keywords:
        if source in text:
            found_sources.append(source)

# dictionary counting without Counter
counts_dict = {}
for item in found_sources:
    if item in counts_dict:
        counts_dict[item] += 1
    else:
        counts_dict[item] = 1
```

In [146…

```
#convert to list of [count, name] and sort
sorting_list = []
for name, count in counts_dict.items():
    sorting_list.append([count, name])

# sort descending by count
sorting_list.sort(reverse=True)
```

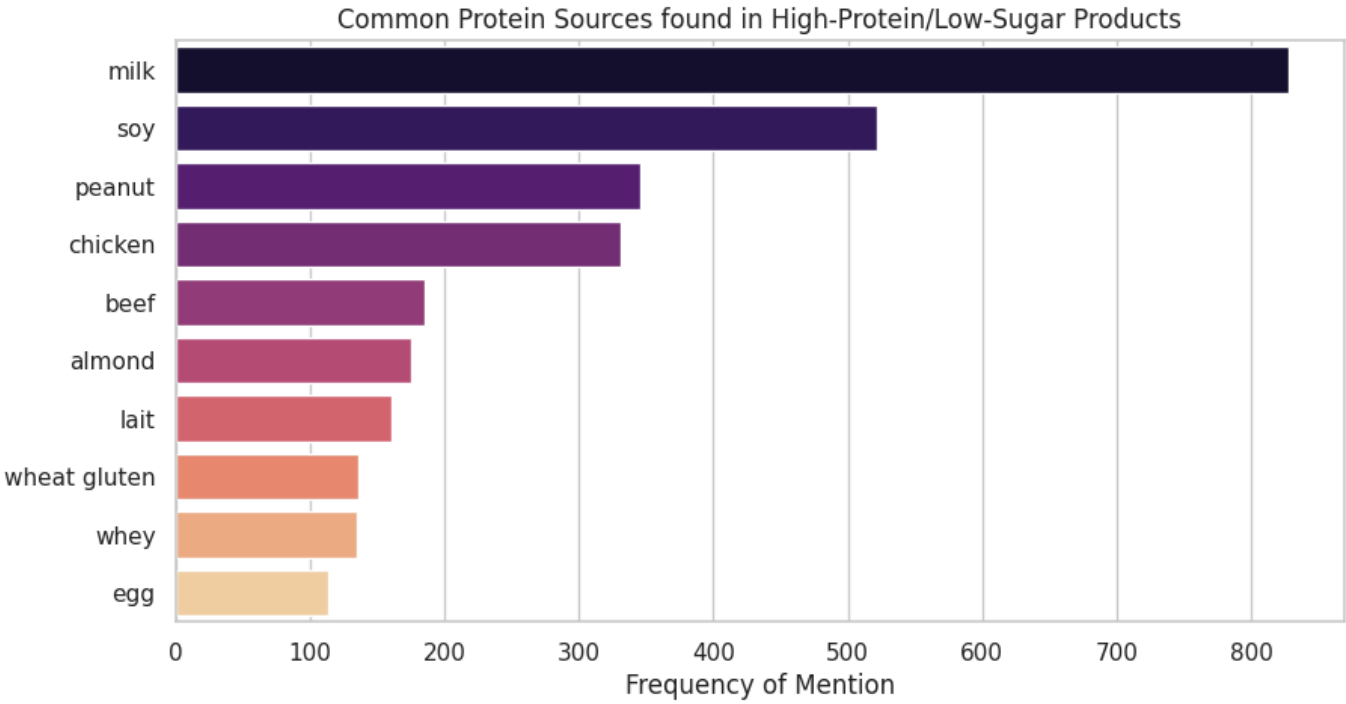
In [147…

```
#extractin top sources
top_sources = []
for i in range(min(3, len(sorting_list))):
    count, name = sorting_list[i]
    top_sources.append((name, count))

print("Top 3 Common Protein Sources in the High Protein Data:")
rank = 1
for item in top_sources:
    source = item[0]
    count = item[1]
    print(f"{rank}. {source.strip().upper()} (found in {count} products)")
    rank += 1
```

```
Top 3 Common Protein Sources in the High Protein Data:
1. MILK (found in 828 products)
2. SOY (found in 522 products)
3. PEANUT (found in 346 products)
```

In [148…

```
# visual representation
plt.figure(figsize=(10, 5))
top_data = sorting_list[:10]
frequencies = [item[0] for item in top_data]
sources = [item[1] for item in top_data]

sns.barplot(x=frequencies, y=sources, hue=sources, palette='magma', legend=False)
plt.title('Common Protein Sources found in High-Protein/Low-Sugar Products')
plt.xlabel('Frequency of Mention')
plt.show()
```



6. The "Candidate's Choice" Challenge

Why I added this:

I chose this because Sugar vs. Protein shows one dimension of health but the Nutriscore grade captures the full nutritional picture i.e sugar, fat, salt, fiber, and protein etc combined into one A to E score. Therefore, category full of D and E grades represents a larger Blue Ocean opportunity for disruption.

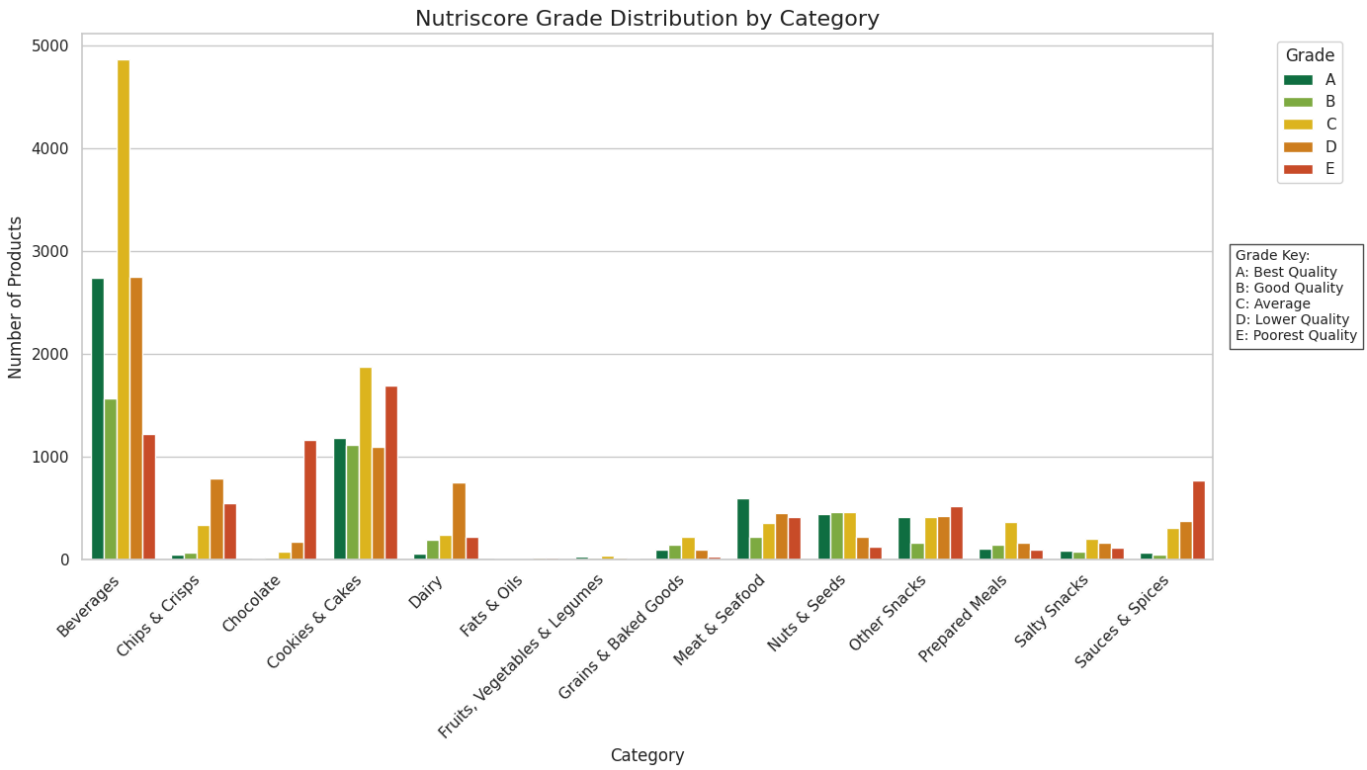
In [149…

```
# only rows with a valid nutriscore grade
nutriscore_data = food_data[food_data['nutriscore_grade'].isin(['a', 'b', 'c', 'd', 'e'])].co

# standardize grades for display
nutriscore_data['nutriscore_grade'] = nutriscore_data['nutriscore_grade'].str.upper()

# counting only products per category per grade
grade_counts = nutriscore_data.groupby(['primary_category', 'nutriscore_grade']).size().reset_

# filtering for categories with significant data (Total > 50 graded products)
category_totals = grade_counts.groupby('primary_category')['product_count'].sum()
significant_categories = category_totals[category_totals > 50].index
grade_counts = grade_counts[grade_counts['primary_category'].isin(significant_categories)]
```

In [150…

```
# grouped bar chart using Matplotlib/Seaborn
import matplotlib.pyplot as plt
import seaborn as sns

# Set default style for consistency
sns.set_theme(style='whitegrid')

# Create a figure and axes
plt.figure(figsize=(14, 8))

# Plot the grouped bar chart
sns.barplot(
    data=grade_counts,
    x='primary_category',
    y='product_count',
    hue='nutriscore_grade',
    palette={'A': '#038141', 'B': '#85BB2F', 'C': '#FECB02', 'D': '#EE8100', 'E': '#E63E11'},
    order=grade_counts['primary_category'].unique() # Maintain category order
```

```
)

plt.title('Nutriscore Grade Distribution by Category', fontsize=16)
plt.xlabel('Category', fontsize=12)
plt.ylabel('Number of Products', fontsize=12)

# Add legend for Nutriscore Grades
plt.legend(title='Grade', bbox_to_anchor=(1.05, 1), loc='upper left')

# Add significance text as an annotation
significance_text = (
    "Grade Key:\n"
    "A: Best Quality\n"
    "B: Good Quality\n"
    "C: Average\n"
    "D: Lower Quality\n"
    "E: Poorest Quality"
)
plt.text(1.02, 0.5, significance_text, transform=plt.gca().transAxes, fontsize=10, verticalal

plt.xticks(rotation=45, ha='right') # Rotate x-axis labels for readability
plt.tight_layout() # Adjust layout to prevent labels from being cut off
plt.show()
```



In [151…

```
#  the percentage of unhealthy (D/E) grades for ALL categories
total_per_category = nutriscore_data.groupby('primary_category').size().reset_index(name='tot
bad_per_category = nutriscore_data[nutriscore_data['nutriscore_grade'].isin(['D', 'E'])].grou

full_health_summary = total_per_category.merge(bad_per_category, on='primary_category', how='
full_health_summary['pct_unhealthy'] = (full_health_summary['bad_count'] / full_health_summar

#srting by the highest percentage of unhealthy products to highlight opportunity areas
full_health_summary = full_health_summary.sort_values('pct_unhealthy', ascending=False)

full_health_summary[['primary_category', 'total', 'pct_unhealthy']]
```

Out[151…

	primary_category	total	pct_unhealthy
4	Chocolate	1421	93.4
3	Chips & Crisps	1763	75.4
15	Sauces & Spices	1553	73.3
2	Candies & Sweets	11	72.7
7	Fats & Oils	51	68.6
6	Dairy	1449	66.8
12	Other Snacks	1912	48.8
0	Bars & Granola	17	47.1
14	Salty Snacks	631	43.7
10	Meat & Seafood	2018	42.5
5	Cookies & Cakes	6940	40.0
1	Beverages	13131	30.2
13	Prepared Meals	851	29.4
8	Fruits, Vegetables & Legumes	94	24.5
11	Nuts & Seeds	1695	20.2
9	Grains & Baked Goods	563	19.7

In [152…

```
!jupyter nbconvert --to html /content/AmaliTech_Capstone_Project.ipynb
```

```
[NbConvertApp] Converting notebook /content/AmaliTech_Capstone_Project.ipynb to html
[NbConvertApp] WARNING | Alternative text is missing on 4 image(s).
[NbConvertApp] Writing 1114064 bytes to /content/AmaliTech_Capstone_Project.html
```